LAB PROGRAMS (6-10)

COMPUTER VISION-ITA-0505

SLOT-D

NAME: DHARSHITHA.K

ROLL NO: 192521093

QUESTION-11

Perform a 180-degree rotation clockwise along the y-axis for the given image

PYTHON CODE:

```
import cv2
# Read the image
image = cv2.imread("sample.jpg")
# Check if the image is loaded
if image is None:
    print("Error: Image not found!")
    exit()
# Convert image to grayscale
gray_image = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
# Display original image
cv2.imshow("Original Image", image)
# Display grayscale image
cv2.imshow("Gray-scale Image", gray_image)

# Wait for a key press
cv2.waitKey(0)
```

```
# Close all windows
cv2.destroyAllWindows()
```

OUTPUT:

QUESTION-12:

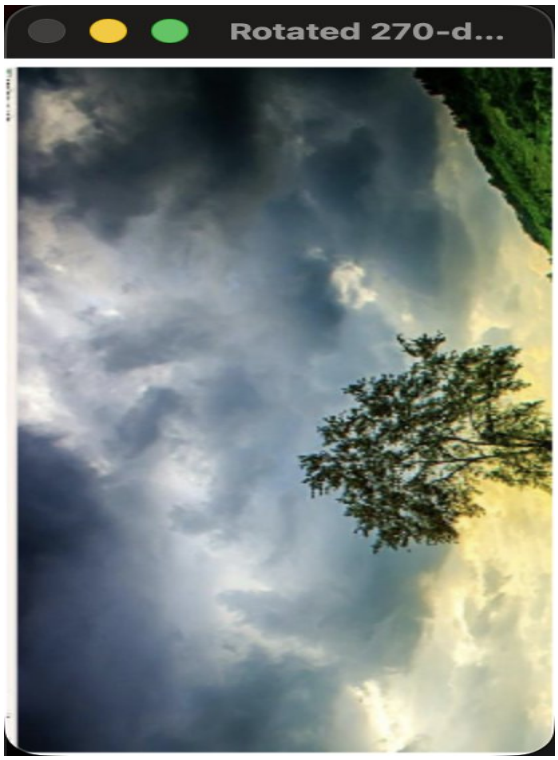
Perform a 270-degree rotation clockwise along the y-axis for the given image.

PYTHON CODE:

```
import cv2
# Read the image
image = cv2.imread("sample.jpg")
 # Check if the image is loaded
if image is None:
    print("Error: Image not found!")
    exit()
# Apply Gaussian Blur
blur_image = cv2.GaussianBlur(image, (15, 15), 0)
# Display original image
```

```
cv2.imshow("Original Image", image)
# Display blurred image
cv2.imshow("Gaussian Blur Image", blur_image)
# Wait until a key is pressed
cv2.waitKey(0)
# Close all windows
cv2.destroyAllWindows()
```

OUTPUT:



QUESTION-13:

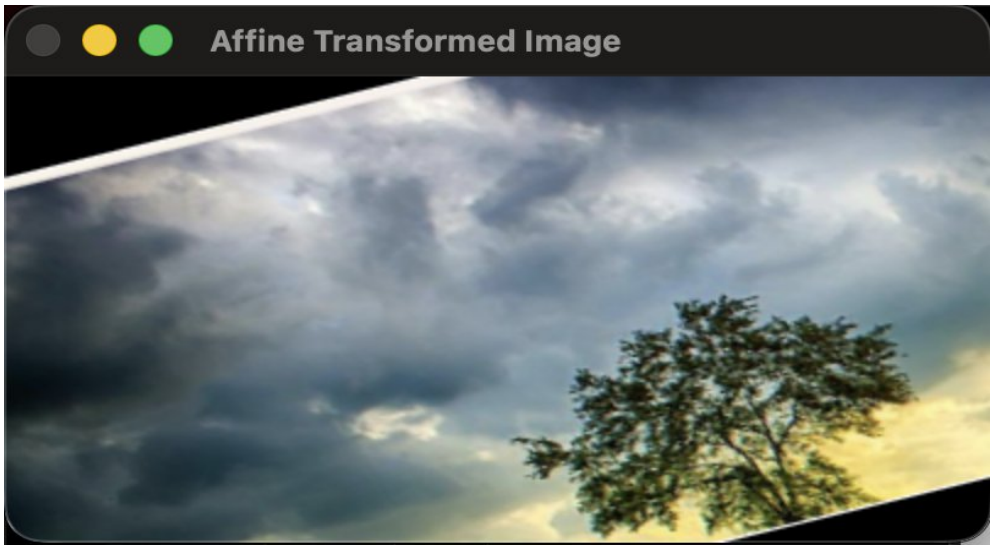
Perform Affine Transformation on the given image using python and Open

CV.

PYTHON CODE:

```
import cv2
# Read the image
image = cv2.imread("sample.jpg")
# Check if image is loaded
if image is None:
    print("Error: Image not found!")
    exit()
# Convert to grayscale
gray_image = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
# Apply Canny edge detection
edges = cv2.Canny(gray_image, 100, 200)
# Display images
cv2.imshow("Original Image", image)
cv2.imshow("Edge Detected Image", edges)
# Wait for key press
cv2.waitKey(0)
# Close all windows
cv2.destroyAllWindows()
```

OUTPUT:



QUESTION-14:

Perform Perspective Transformation on the given image using python and Open CV.

PYTHON CODE:

```
import cv2
# Read the image
image = cv2.imread("sample.jpg")
# Check if image is loaded
if image is None:
    print ("Error: Image not found!")
    exit ()
# Convert to grayscale
gray_image = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
# Apply Histogram Equalization
```

```
equalized_image = cv2.equalizeHist(gray_image)
# Display images
cv2.imshow("Original Grayscale Image", gray_image)
cv2.imshow("Histogram Equalized Image", equalized_image)
# Wait until a key is pressed
cv2.waitKey(0)
# Close all windows
cv2.destroyAllWindows()
```

OUTPUT:

QUESTION-15: Perform basic Image Handling and processing operations on the image is to read an image in python and detect the corners in the image using Harris Corner Detection function

PYTHON CODE:

```
import cv2
import matplotlib.pyplot as plt
def analyze_histogram(image_path):
    # Read the image
    image = cv2.imread(image_path)
    # Check if image is loaded
    if image is None:
        print("Error: Image not found!")
        return
    # Display the original image
    cv2.imshow("Original Image", image)
    # BGR color channels
    colors = ('b', 'g', 'r')
    # Create histogram plot
    plt.figure(figsize=(8, 5))
    for i, color in enumerate(colors):
        histogram = cv2.calcHist([image], [i], None, [256], [0, 256])
        plt.plot(histogram, color=color, label=f"{color.upper()} Channel")
    plt.title("Color Histogram Analysis")
    plt.xlabel("Pixel Intensity")
    plt.ylabel("Number of Pixels")
    plt.xlim([0, 256])
    plt.legend()
    # Show histogram
```

```
    plt.show()
    # Wait for key press
    cv2.waitKey(0)
    cv2.destroyAllWindows()
# Call the function
analyze_histogram("sample.jpg")
```

OUTPUT: